

HTTP Request Smuggling in the Multiverse of Parsing Flaws

HTTP Request Smuggling in the Multiverse of Parsing Flaws - Zeyu Zhang, Zeyu Zhang.

- Published: 2022
- Original: <<https://infosec.zeyu2001.com/2022/http-request-smuggling-in-the-multiverse-of-parsing-flaws>>
- Preserved from: <https://infosec.zeyu2001.com/2022/http-request-smuggling-in-the-multiverse-of-parsing-flaws> (manual-import) on 2026-09-29
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

For the complete documentation index, see [llms.txt](#). Markdown versions of documentation pages are available by appending `.md` to page URLs; this page is available as [Markdown](#).

HTTP Request Smuggling in the Multiverse of Parsing Flaws

Nowadays, novel HTTP request smuggling techniques rely on subtle deviations from the HTTP standard. Here, I discuss some of my recent findings and novel techniques.

Some time earlier this year, I conducted a bit of independent research on HTTP request smuggling and found a couple of vulnerabilities.

This article expands on my talk on the topic at [BSides Singapore](#) 2022.

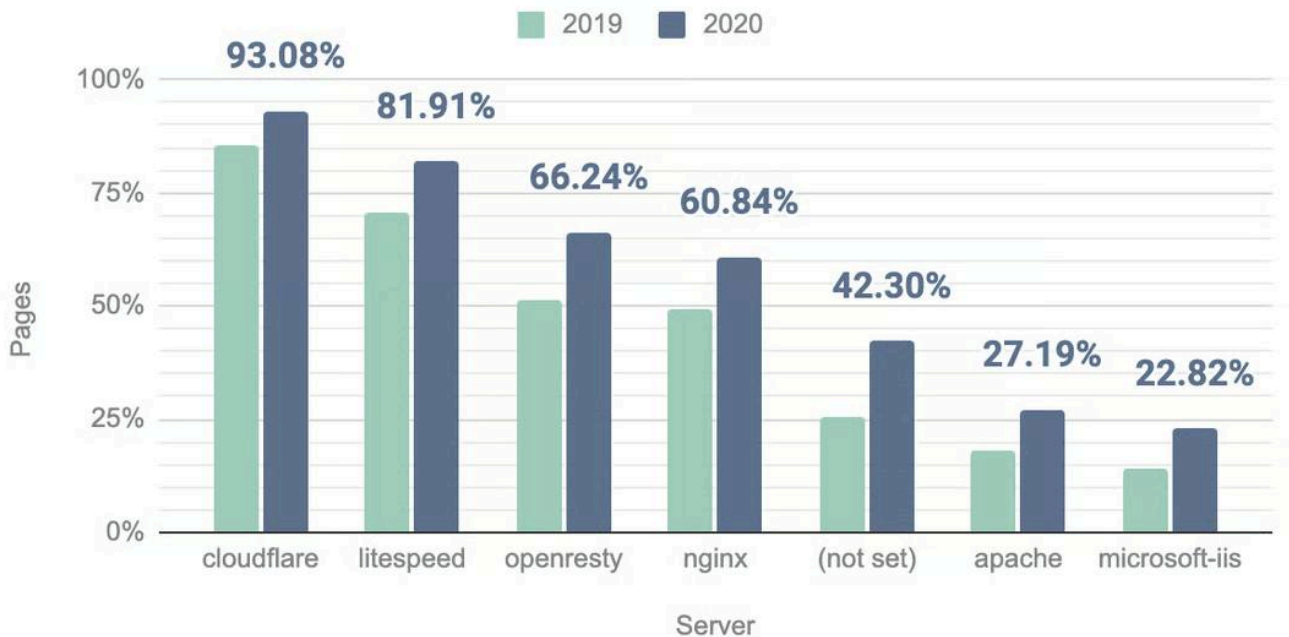
What is HTTP Request Smuggling?

To understand HTTP request smuggling, we have to first take a trip down memory lane.

The HTTP protocol has undergone several changes since its inception, and the latest protocol version is HTTP/3. While HTTP/2 is the most popular version today, HTTP/1.x still comprises a significant amount of web traffic and is crucially important in understanding HTTP request smuggling.

HTTP/2 usage by server

Web Almanac 2020: HTTP/2



The major difference between HTTP/1.x and HTTP/2 is the fact that HTTP/2 evolved from a purely text-based protocol to a binary protocol. In HTTP/1.x, the `Content-Length` and `Transfer-Encoding` headers determined the length of an HTTP request. It is this reliance on two special headers that enabled the earliest discoveries of HTTP request smuggling.

A Brief History

	Protocol Implementation	Connection Reuse	Length Determined By
HTTP/1.0	• Purely text-based	✗	• Content-Length header • Transfer-Encoding chunk size
HTTP/1.1		✓	
HTTP/2	• Binary protocol		• Length field built into protocol

BSides Singapore 2022

BSidesSG

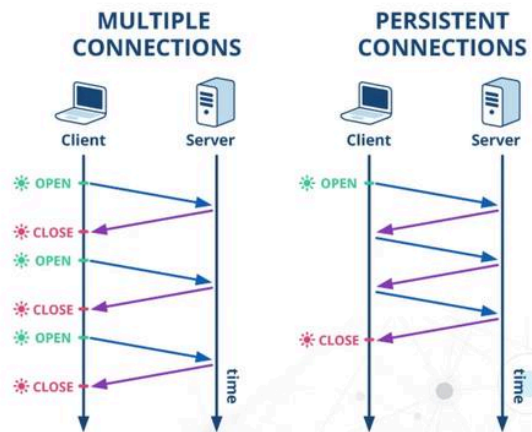
BSIDES SINGAPORE

But this alone is not enough. In HTTP/1.0, one TCP connection is used for each HTTP request - any two HTTP requests cannot interfere with each other. With HTTP/1.1 came along, the concept of persistent connections was introduced. This introduced an entirely

new vector of attack - if one request earlier in the TCP stream could interfere with another downstream request, a variety of vulnerabilities could occur.

Connection Reuse

- HTTP/1.0 used **one connection per request**
- Different requests could not interfere with each other
- HTTP/1.1 allowed for **persistent connections**, allowing the same TCP connection to be **re-used between requests**
- This allowed different requests to interfere with each other!
- HTTP request smuggling makes use of this to **“poison” the TCP stream**



BSides Singapore 2022

BSidesSG

This becomes particularly relevant when considering architectures comprising a frontend proxy (such as Nginx, Apache HTTP Server and HAProxy) with backend web servers. Consider the following example.

“Vanilla” Request Smuggling – CL.TE

```
GET / HTTP/1.1
Host: example.com
Content-Length: 53
Transfer-Encoding: chunked
```

0

```
GET /internal HTTP/1.1
Host: example.com
```

Hmm... I have a request to / that contains a 53-byte body. I'm sure that the backend server would agree!

Frontend

```
GET / HTTP/1.1
Host: example.com
Content-Length: 53
Transfer-Encoding: chunked
```

0

```
GET /internal HTTP/1.1
Host: example.com
```

Cool, I have a request with an empty chunked body, followed by a second request to /internal.

Backend

BSides Singapore 2022

BSidesSG

The frontend proxy parses the `Content-Length` header, forwarding the `GET /internal` request as part of the 53-byte request body.

The backend web server, on the other hand, parses the `Transfer-Encoding: chunked` header and interprets the first request to end at the `0` chunk size. This meant that from the perspective of the backend server, there are two requests - one to `/` and one to `/internal`.

Note that in this case, the backend is spec-compliant and the frontend proxy is not. According to [RFC7230 section 3.3.3](#), the `Transfer-Encoding` header overrides the `Content-Length`.

If a message is received with both a `Transfer-Encoding` and a `Content-Length` header field, the `Transfer-Encoding` overrides the `Content-Length`. Such a message might indicate an attempt to perform request smuggling (Section 9.5) or response splitting (Section 9.4) and ought to be handled as an error. A sender **MUST** remove the received `Content-Length` field prior to forwarding such a message downstream.

Enter the Multiverse

This section would be split into different groups of parsing flaws. Because I often compile multiple issues into a single report, the resulting CVEs comprise of multiple issues. It is more meaningful to discuss the various types of issues rather than each CVE individually.

Some Observations

When I was looking into various web servers and proxies, I noticed some things that I would like to point out.

First, it seems like lots of research has been done on web proxy technologies, but not a lot has been done on backend servers. This is also reflected in the relative security of projects like Nginx and HAProxy against request smuggling. It is important to note that in most cases, a request smuggling attack reveals a two-pronged issue that requires both the frontend proxy and the backend server to be somewhat non-compliant.

This sometimes makes it difficult to demonstrate impact when disclosing vulnerabilities, as the impact often has to be qualified with a precondition that some other server in the stack is also non-compliant. It is important for maintainers not to dismiss request smuggling vectors as low impact or insignificant just because of this.

Second, most "traditional" request smuggling techniques have been patched. These are techniques that have been popularly taught and demonstrated, for example:

- Duplicate `Content-Length` headers (CL.CL)
- Frontend server uses `Content-Length`, backend uses `Transfer-Encoding` (CL.TE)
- Frontend server uses `Transfer-Encoding`, backend uses `Content-Length` (TE.CL)

The next part of this article will discuss *subtle* deviations from the HTTP standard that can lead to request smuggling. These are vectors that may seem trivial but are often neglected.

Last, when implementing [RFC7230](#), often the `SHOULD` clauses are equally important in preventing HTTP request smuggling. Sometimes differences in interpreting such clauses can lead to disagreements between servers.

Number Parsing Flaws

According to the RFC, the `Content-Length` value comprises of any number of `DIGIT` s.

```
Content-Length = 1*DIGIT
...
Any Content-Length field value greater than or equal to zero is valid.
```

A `DIGIT` in the ABNF standard consists of strictly 0-9 only. However, due to number parsing implementations, many parsers will accept non-conformant values like `+23`.

Consider the following requests.

```
GET / HTTP/1.1
Content-Length: +23

GET / HTTP/1.1
Dummy: GET /forbidden HTTP/1.1
```

In a previous version of [Apache Traffic Server](#), the `+23` content length is silently ignored, and the first request is interpreted as having zero content length.

Many web servers, however, will interpret `+23` as a valid content length. This means that the requests will now be interpreted very differently. The first request has a 23-byte body, ending at the `Dummy` header.

```
GET / HTTP/1.1
Content-Length: +23

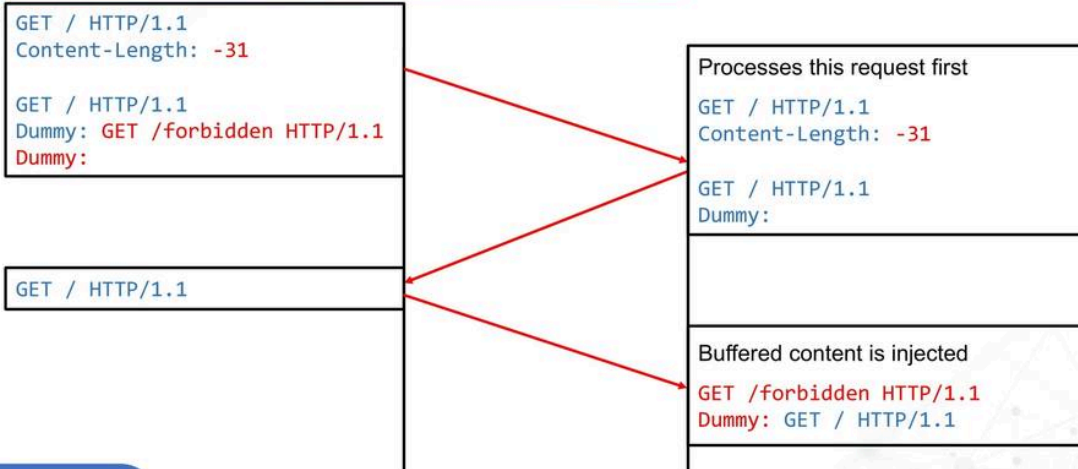
GET / HTTP/1.1
Dummy:
```

The second request will now instead be routed to `/forbidden`.

```
GET /forbidden HTTP/1.1
```

This starts to get more interesting when negative numbers are involved. For example, the following was the behaviour of [Twisted Web](#) when encountering negative content lengths.

Number Parsing Flaws



BSides Singapore 2022

BSidesSG



Chunk sizes also present a similar issue - servers should not accept the `0x` prefix. Because of differences in parsing hexadecimal numbers, this simple request can be interpreted differently.

```
GET / HTTP/1.1
Transfer-Encoding: chunked

0x12
GET / HTTP/1.1

0
```

Some parsers will simply parse the number up to the first non-hex digit. This leads to the early termination of the request and consequently the smuggling of a second request.

```
GET / HTTP/1.1
Transfer-Encoding: chunked

0
GET / HTTP/1.1

0
```

We can see how language-specific behaviour plays a part in these scenarios. In fact, the behaviour of the Python-based servers was in line with how `int()` handles integer strings,

and Puma's behaviour was in line with Ruby's `to_i` (which parses integer strings up to the first non-decimal character).

Summary

CVE ID	Server (Language)	Behavior
CVE-2022-24761	Waitress (Python)	Accept 'signed' ($\pm$) and 0x-prefixed Content-Length and chunk sizes
CVE-2022-24801	Twisted (Python)	Accept 'signed' ($\pm$) and 0x-prefixed Content-Length and chunk sizes
CVE-2022-24790	Puma (Ruby)	<code><p>abc → 0</p><p>99 balloons → 99</p></code>

Whitespace is More Than 0x20

Headers allow for optional whitespace (`OWS`) before and after the field values.

```
OWS = *( SP / HTAB )
header-field = field-name ":" OWS field-value OWS
```

Importantly, only two whitespace characters are considered valid here - space and horizontal tab. But this definition of whitespace is often incompatible with that of generic stripping functions in most programming languages.

Consider the following request. If a proxy were to interpret the transfer coding as `\rchunked`, this may be interpreted as an invalid encoding and ignored.

Whitespace is More Than 0x20

```
POST / HTTP/1.1
Host: example.com
Transfer-Encoding: \rchunked
```

Proxy **ignores** invalid Transfer-Encoding value `\rchunked`

```
DELETE / HTTP/1.1
Host: example.com
Content-Length: 23
Padding:
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
```

Second request includes 23-byte body GET /admin HTTP/1.1

23 GET /admin HTTP/1.1

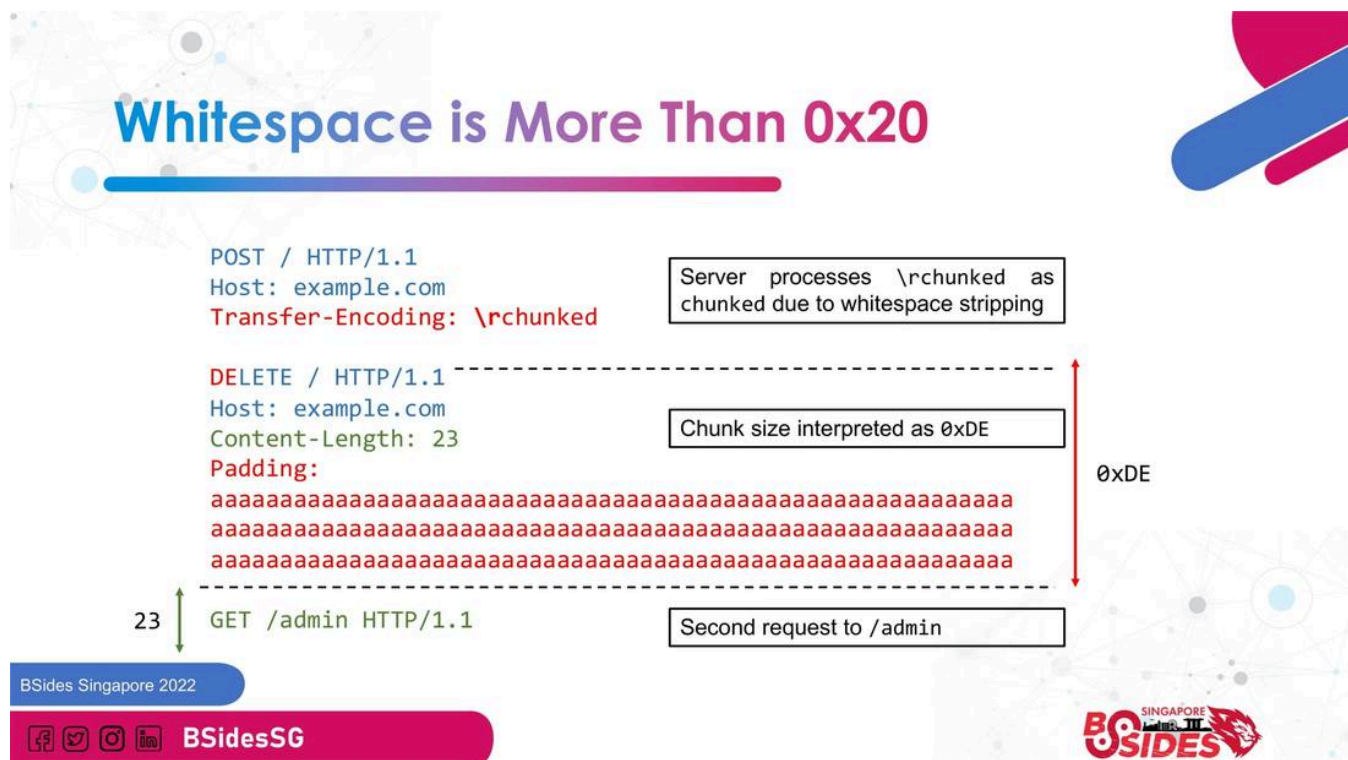
BSides Singapore 2022

BSidesSG

SINGAPORE BSIDES

The second request would then contain a 23-byte body including `GET /admin`.

But a server that incorrectly strips the `\r` character from the `Transfer-Encoding` header would not see it the same way.



A much more classic technique involves whitespace between the header names and colon. By stripping the header names of whitespace, headers like `Content-Length : 5` were allowed in [mitmproxy](#). This particular case is clearly addressed in the RFC.

No whitespace is allowed between the header field-name and colon. In the past, differences in the handling of such whitespace have led to security vulnerabilities in request routing and response handling. A server MUST reject any received request message that contains whitespace between a header field-name and colon with a response code of `400` (Bad Request). A proxy MUST remove any such whitespace from a response message before forwarding the message downstream.

Summary

CVE ID	Server (Language)	Behavior
CVE-2022-28129	Apache Traffic Server	<code>Content-Length[\x0b]: 0</code> accepted
CVE-2022-24766	mitmproxy (Python)	<code>Content-Length[SP]: X</code> accepted
CVE-2022-1705	net/http (Golang)	<code>Transfer-Encoding: \rchunked</code> accepted

Transfer-Encoding - You Had One Job

A quick primer on the `Transfer-Encoding` header - encodings are stated from first to last, so `gzip, chunked` would mean that the decoding server needs to decode the `chunked` body

as `gzip` data.

According to RFC 7230, `chunked` must be the final value in the `Transfer-Encoding` header.

If a `Transfer-Encoding` header field is present in a request and the chunked transfer coding is not the `final` encoding, the message body length cannot be determined reliably; the server **MUST** respond with the `400` (Bad Request) status code and then close the connection.

But the deprecated RFC 2616 actually allows the `identity` encoding, which means "the use of no transformation whatsoever". In fact, in this RFC, the `chunked` transfer-coding is only used when the `Transfer-Encoding` value is not `identity`.

If a `Transfer-Encoding` header field (section 14.41) is present and has any value other than `"identity"`, then the transfer-length is defined by **use** of the `"chunked"` transfer-coding (section 3.6), unless the message is terminated by closing the connection.

Puma, in particular, assumed the opposite - as long as *any* of the `Transfer-Encoding` values is `chunked`, the message is parsed with chunked encoding. This means that the following request is considered `chunked`, although the final transformation is `identity`.

```
GET / HTTP/1.1
Host: example.com
Transfer-Encoding: chunked, identity
```

Up till recently, many major proxies still supported the `identity` transfer-coding. This meant that any of these proxies used in combination with Puma would have allowed for request smuggling through the above request.

It is also important to reject any invalid `Transfer-Encoding` value. Servers often accept invalid values due to parsing flaws, and silently ignoring these malformed transfer-codings opens the door to request smuggling. When no supported `Transfer-Encoding` values are found, Puma would silently ignore the header altogether.

SEETF 2022 Challenge – ATS x Puma

```
POST / HTTP/1.1
Host: example.com
Transfer-Encoding: "chunked"
```

Apache Traffic Server accepts
"chunked" as chunked

```
DELETE / HTTP/1.1
Host: example.com
Padding:
```

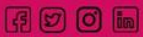
```
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
```

```
0: x
```

POST request includes 0xDE byte
body

0xDE

BSides Singapore 2022



BSidesSG



This is a good example of how research on web servers is equally important to that on web proxies. While the argument could be made that the fault lies with Apache Traffic Server for accepting the malformed "chunked" value, the attack would not have been possible if Puma threw a 400 Bad Request when encountering it.

SEETF 2022 Challenge – ATS x Puma

```
POST / HTTP/1.1
Host: example.com
Transfer-Encoding: "chunked"
```

Puma silently ignores the invalid
Transfer-Encoding

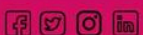
```
DELETE / HTTP/1.1
Host: example.com
Padding:
```

```
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
```

```
0: x
```

DELETE request interpreted as a
second, separate request

BSides Singapore 2022



BSidesSG



Because of the variability of the Transfer-Encoding header, the parsing behaviour of various servers when it comes to this header is quite interesting. In particular, I noted an interesting behaviour in the Node.js http module.

Transfer-Encoding – You Had One Job...

```
501 //
502 // Transfer-Encoding
503 //
504
505 n('header_value_te_chunked')
506   .transform(p.transform.toLowerUnsafe())
507   .match(
508     'chunked',
509     n('header_value_te_chunked_last'),
510   )
511   .otherwise(n('header_value_te_token'));
512
513 n('header_value_te_chunked_last')
514   .match(' ', n('header_value_te_chunked_last'))
515   .peek([ '\r', '\n' ], this.update('header_state',
516     HEADER_STATE.TRANSFER_ENCODING_CHUNKED,
517     'header_value_otherwise'))
518   .otherwise(n('header_value_te_chunked'));
519 -
520
521 n('header_value_te_token')
522   .match(',', n('header_value_te_token_ows'))
```

Matches chunked then CRLF,
otherwise match chunked again?

```
518 //
519 // Transfer-Encoding
520 //
521
522 n('header_value_te_chunked')
523   .transform(p.transform.toLowerUnsafe())
524   .match(
525     'chunked',
526     n('header_value_te_chunked_last'),
527   )
528   .otherwise(n('header_value_te_token'));
529
530 r_value_te_chunked_last')
531 (' ', n('header_value_te_chunked_last'))
532 [ '\r', '\n' ], this.update('header_state',
533   ER_STATE.TRANSFER_ENCODING_CHUNKED,
534   'header_value_otherwise'))
535 +
536 + .peek(',', forbidAfterChunkedInRequest(n('header_value_te_chunked')))
537 + .otherwise(n('header_value_te_token'));
538
539 n('header_value_te_token')
540   .match(',', n('header_value_te_token_ows'))
```

BSides Singapore 2022

BSidesSG



In the original code, when `chunked` is matched, a check is made to see if `chunked` is the final encoding. If a CRLF sequence is encountered, `chunked` is taken to be the final encoding, and the request body will be parsed as chunked. Otherwise, it attempts to match `chunked` again.

But this logic forgets to look for a `,` separator if the CRLF sequence is not found, meaning that the following is a valid chunked request.

GET / HTTP/1.1

Host: example.com

Transfer-Encoding: chunkedchunked

...

Summary

CVE ID	Server (Language)	Behavior
CVE-2022-24766	Puma (Ruby)	- Does not check that chunked is the final encoding - Silently ignores invalid encodings
CVE-2022-1705	http (Node.js)	Accepts malformed encodings, e.g. <code>chunkedchunked</code>

obs-fold - Not So Obsolete

Historically, multi-line headers were allowed by starting each extra line with either a space or horizontal tab. RFC 7230 deprecates such line-folding (`obs-fold`).

```
field-value    = *( field-content / obs-fold )
obs-fold      = CRLF 1*( SP / HTAB )
```

For backwards compatibility, `obs-fold` is supported by most servers. This is spec-compliant.

A server that receives an `obs-fold` in a request message that is not within a message/http container MUST either reject the message by sending a `400` (Bad Request), preferably with a representation explaining that obsolete line folding is unacceptable, or replace each received `obs-fold` with one or more SP octets prior to interpreting the field value or forwarding the message downstream.

The trouble begins when implementing the rest of the spec while supporting `obs-fold`. As we saw above, one assumption made by the Node.js parser was that the `Transfer-Encoding` header would end when encountering the CRLF sequence - `chunked` followed by CRLF would mean that the transfer-coding is `chunked`.

This makes sense until we consider that the parser also supports `obs-fold`, so the following multi-line header would be interpreted wrongly.

```
GET / HTTP/1.1
Host: example.com
Transfer-Encoding: chunked
[SP], identity
```

Instead of parsing the transfer-coding as `identity`, `chunked` is used instead.

Summary

CVE ID	Server (Language)	Behavior
CVE-2022-32215	http (Node.js)	Early termination of multi-line <code>Transfer-Encoding</code> headers.

Bonus: LF vs. CRLF

This discussion was not included in my talk because this is a slightly more contested topic and it is sometimes ambiguous whether this is a legitimate issue.

```
GET / HTTP/1.1
Dummy: x[\n]Content-Length: 23

GET / HTTP/1.1
Dummy: GET /forbidden HTTP/1.1
```

Note that each line above is delimited by the CRLF sequence.

If a proxy strictly delimits each line by CRLF and incorrectly allows the `\n` character as a valid character in header values, a backend that delimits each line by only a bare LF will interpret the requests as

```
GET / HTTP/1.1
Dummy: x
Content-Length: 23

GET / HTTP/1.1
Dummy: GET /forbidden HTTP/1.1
```

While this seems dangerous, the spec actually allows for a single LF to be used to delimit lines, albeit in a `MAY` clause.

Although the line terminator **for** the start-line and header fields is the sequence CRLF, a recipient `MAY` recognize a single LF as a line terminator and ignore any preceding CR.

Some servers like Waitress and Node.js have taken this potential vector into consideration and switched to the most-spec-compliant method of delimiting lines with the CRLF sequence.

Bonus: Other Findings

There are some individual findings that didn't fit into any of the above groups but are interesting to discuss nonetheless. This discussion was not included in my talk for brevity.

Puma - Duplicate Content-Length Headers (CL.CL)

This one is a relatively common technique. Puma allowed multiple `Content-Length` headers.

```
Content-Length: 0
Content-Length: 5
```

Note that internally, this will result in a final `Content-Length` value of `0, 5`, but Ruby's `to_i` function will stop parsing at the first non-decimal character, and therefore the first `Content-Length` header is used to determine the request length. This is non-compliant.

If a message is received without Transfer-Encoding and with either multiple `Content-Length` header fields having differing field-values or a single `Content-Length` header field having an invalid value, then the message framing is invalid and the recipient **MUST** treat it as an unrecoverable error. **If this** is a request message, the server **MUST** respond with a `400` (Bad Request) status code and then close the connection.

If an upstream proxy processes the second `Content-Length` header instead, request smuggling attacks can occur.

Node.js - Whitespace Before First Header

This one is quite interesting. According to the RFC, whitespace between the start-line and the first header field is not allowed. It even explicitly mentions the associated security risks.

A sender MUST NOT send whitespace between the start-line and the first header field. A recipient that receives whitespace between the start-line and the first header field MUST either reject the message as invalid or consume each whitespace-preceded line without further processing of it (i.e., ignore the entire line, along with any subsequent lines preceded by whitespace, until a properly formed header field is received or the header section is terminated).

The presence of such whitespace in a request might be an attempt to trick a server into ignoring that field or processing the line after it as a `new` request, either of which might result in a security vulnerability `if` other implementations within the request chain interpret the same message differently. Likewise, the presence of such whitespace in a response might be ignored by some clients or cause others to cease parsing.

Node.js allowed whitespace in this location, leading to some potentially interesting vectors. In the following request, the content length header name is taken to be the literal string `"Content-Length"` and different from the normal `"Content-Length"` header. It is therefore not indicative of the request body.

```
GET / HTTP/1.1
[SP]Content-Length: 23
Foo: Bar

GET / HTTP/1.1
Dummy: GET /forbidden HTTP/1.1
```

If a frontend proxy parses the malformed `Content-Length` header, smuggling attacks can occur. However, I have yet to find a proxy that exhibits such behaviour - most will correctly reject the request as per the RFC.

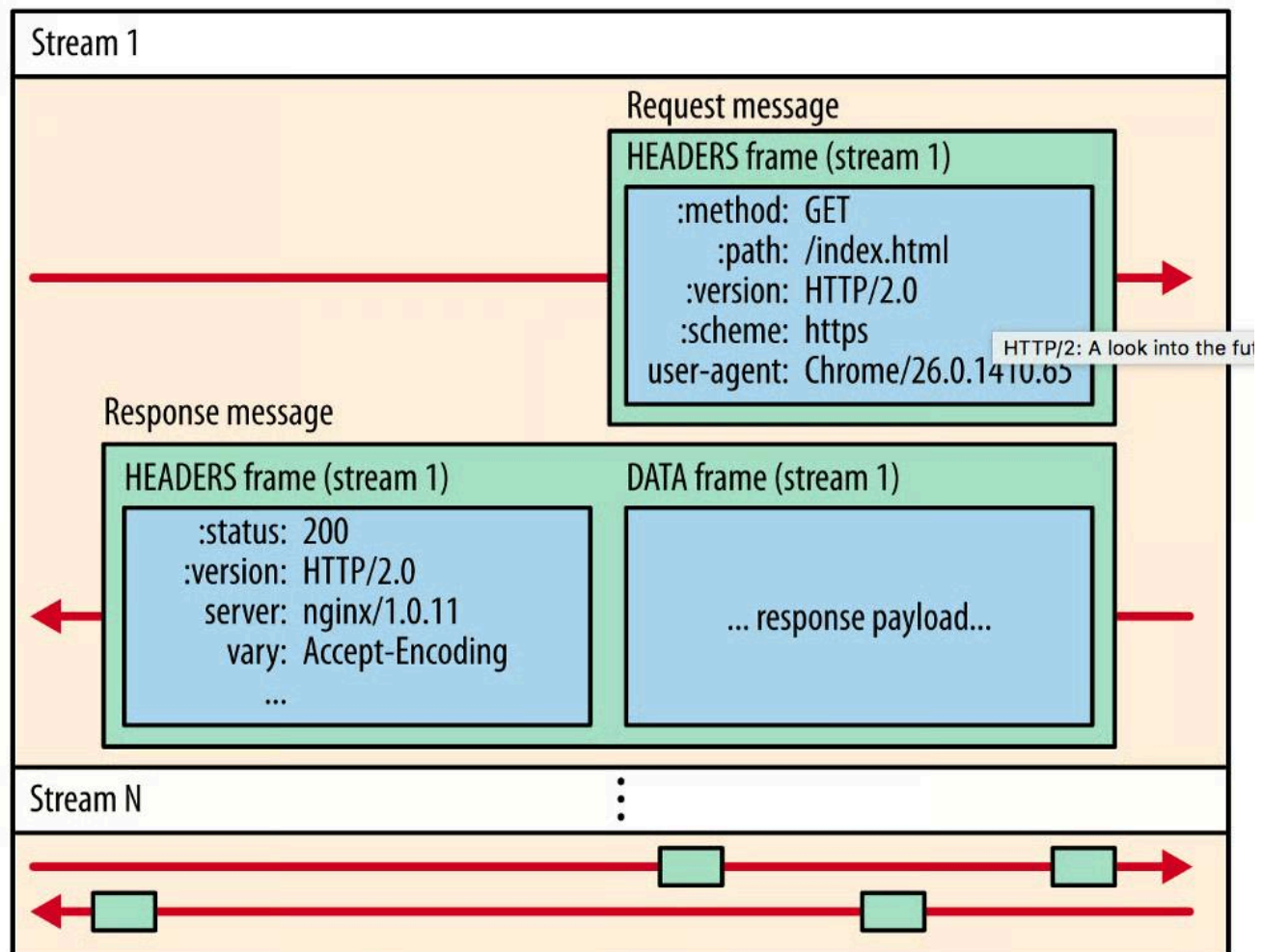
Since there was limited demonstrable impact, this was handled by the Node.js team as a [public issue](#).

14 Million Futures

HTTP/2 Request Smuggling

Previously we discussed how HTTP/2 uses a binary protocol, instead of the text-based one that HTTP/1.x used. Each HTTP/2 data frame now had an associated length field built into the protocol, ensuring that there is no ambiguity in HTTP/2 request body lengths.

Connection

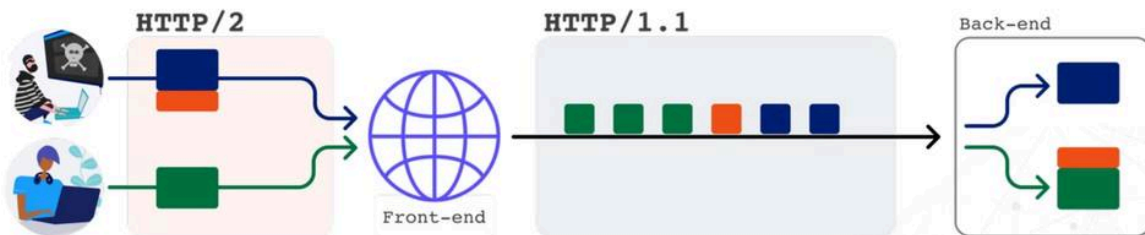


While this sounds good on paper, taking a closer look at the type of architecture required for request smuggling attacks reveals that many of our old techniques are still relevant here.

Even if HTTP/2 is used between the client and frontend proxy, there is no real reason to use HTTP/2 between the proxy and its backend servers. This means that HTTP/2 requests are often *rewritten* to HTTP/1.x before being forwarded to HTTP/1.x backend servers.

HTTP/2 Request Smuggling – How it Started

- HTTP/2 is used between the client and the frontend proxy
- The frontend proxy **downgrades the request to HTTP/1.1** before forwarding them to the backend
- Smuggling vectors leverage the **HTTP/1.1 Content-Length and Transfer-Encoding** headers



BSides Singapore 2022

BSidesSG



One interesting consequence of this was that since the CRLF sequence was no longer used to delimit request lines in HTTP/2, we could potentially perform CRLF injection on the downgraded HTTP/1.1 request by simply supplying these characters in a HTTP/2 header.

I came across one interesting application of this in [Apache Traffic Server](#). By injecting the CRLF sequence in the HTTP/2 headers frame, we could inject new headers into the rewritten HTTP/1.1 request. More broadly, we could also modify everything below the injection point, including the request body.

HTTP/2 Request Smuggling – How it's Going

```
:scheme: https
:method: GET
:path: /
:authority: localhost
foo: bar\r\n
Content-Length: 4\r\n
\r\n
GET / HTTP/1.1\r\n
authorization: secret

Some content
```

→

```
GET / HTTP/1.1
foo: bar
Content-Length: 4
Host: localhost
Client-ip: 172.19.0.1
X-Forwarded-For: 172.19.0.1
Via: https/2 ... (ApacheTrafficServer/9.1.2)
Transfer-Encoding: chunked

3a
GET / HTTP/1.1

Authorization: secret
```

- Apache Traffic Server reflects the CRLF sequence into the downgraded HTTP/1.1 request
- We could modify everything below the injection point

BSid

BSidesSG



While header injection can be sufficient to cause smuggling attacks, I noticed an interesting aspect of this particular vulnerability. Any headers added *below* our injection point could be forced into the request body by injecting the double-CRLF sequence.

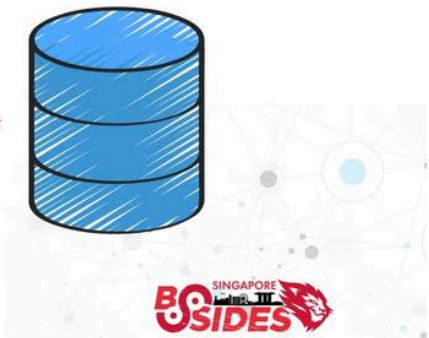
Consider a request that stores the request body that can be later recovered. Sensitive headers being pushed into the request body might lead to information leakage, depending on the application logic.

HTTP/2 Request Smuggling – How it's Going

```
POST /store HTTP/1.1
foo: bar
Injected: Header
Host: localhost
Client-ip: 172.19.0.1
X-Forwarded-For: 172.19.0.1
Via: https/2 ... (ApacheTrafficServer/9.1.2)
Transfer-Encoding: chunked

39
Injected body
Authorization: secret
Some content
0
```

Headers "pushed" into the request body may be stored by backend application



BSides Singapore 2022

BSidesSG

BSIDES SINGAPORE

Summary

CVE ID	Server (Language)	Behavior
CVE-2022-25763	Apache Traffic Server	CRLF injection when downgrading from HTTP/2 to HTTP/1.1

Client-Side Attacks

Just as I'm writing this, new research has been released on [client-side desync attacks](#). I found this new development particularly interesting because it forces a paradigm shift on how we approach smuggling attacks. Client-side desync does not require a proxy-server architecture, only a browser and a single web server.

It would be interesting to see how the community will build on this research to find new and interesting discoveries.